

Worksheet — 1.1 Processors, Input, Output & Storage — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

1. **Key-term match** 1–C, 2–G, 3–A, 4–F, 5–B, 6–E, 7–D, 8–H.

2. **Fill in the blanks (FDE)** ...copied from the **PC** into the **MAR**. A read signal is sent on the **control** bus, and the instruction returns along the **data** bus into the **MDR**. The processor then **increments** the PC... copied from the **MDR** into the **CIR**. Decode: splits the instruction into the **opcode** (the operation) and the **operand** (the data or **address**). Execute: a calculation is performed by the **ALU**, result stored in the accumulator.

3. **State the register** a) PC b) MAR c) MDR d) CIR e) ACC

4. **Put the FDE steps in order** 1 — The address in the PC is copied into the MAR. 2 — Read signal placed on the control bus; instruction returns via the data bus into the MDR. 3 — The PC is incremented ($PC := PC + 1$). 4 — The contents of the MDR are copied into the CIR. *(Steps 3 and 4 may be quoted either way round; the key points are that the PC must be incremented and the MDR must be copied into the CIR.)*

5. **Short answer — performance & architectures** a) Any two, e.g.: - **Clock speed** — pulses per second synchronise the CPU; a higher clock speed means more FDE cycles per second (limited by heat/power). - **Cores** — each core has its own FDE cycle, so multiple instructions can be processed simultaneously. - **Cache** — small, very fast memory on/near the CPU holding frequent/recent data, reducing slow fetches from RAM. - **Pipelining** — overlaps fetch/decode/execute of consecutive instructions, increasing throughput. b) Extra cores only help **if the task can be split into parallel parts and the software is written to use multiple cores**; inherently sequential tasks gain little. c) **Von Neumann** uses one shared memory and bus for both data and instructions (simpler/cheaper but causes a bottleneck because it cannot fetch an instruction and read/write data at the same time); **Harvard** uses separate memories/buses for data and instructions, allowing simultaneous access. Use: Von Neumann — general-purpose PCs/laptops; Harvard — embedded systems / microcontrollers / DSPs. d) Any two, e.g.: CISC has many complex, variable-length, multi-cycle instructions with complexity in hardware; RISC has few simple fixed-length instructions aiming for one per cycle with complexity in the compiler. RISC pipelines more easily and uses less power (good for battery devices/ARM); CISC is typical of x86 desktops. e) A GPU has thousands of small cores that apply the **same operation to many data items in parallel (data parallelism)**, which suits pixels/vertices and large ML matrices. Branch-heavy, dependency-laden or inherently sequential work cannot be parallelised this way (each step needs the previous result), so a CPU handles it better.

6. **Storage device selection table** (example answers)

Type	How it works	One pro	One con	Example use
Magnetic	Magnetised patterns/polarity on a spinning platter or tape, read/written by a R/W head	Huge capacity, low cost per GB	Slow (moving parts), fragile	Bulk/backup storage; tape archive
Flash/SSD	EEPROM cells trap electrons; no moving parts	Fast, silent, low power, durable	Costlier per GB, finite write cycles	Laptop main drive; USB stick; SD card
Optical	Laser detects pits and lands via reflected light	Very cheap, portable, long shelf life	Low capacity, slow, easily scratched	Distributing films/software; long-term disc archive

7. Challenge box (i) Virtual memory uses **secondary storage (a swap/paging file) as extra RAM** when physical RAM is full; pages are moved out to disk and back as needed. It is being used because the open applications need more memory than the installed RAM provides. (ii) The problem is **disk thrashing** — excessive paging. It is slow because **secondary storage (disk) is far slower than RAM**, so constantly swapping pages in and out wastes most of the CPU's time waiting. (iii) e.g. **Install more RAM** so fewer pages need swapping to disk (less/no paging), or **fit an SSD** so any paging that does occur is much faster than on an HDD.

8. Scenario table (justification must be tied to the scenario; example answers)

Scenario	Choice	Justification
a) Washing machine controller	Harvard-architecture microcontroller	The fixed program can sit in separate (read-only) instruction memory while data uses its own memory/bus, allowing simultaneous access — typical of embedded systems.
b) ML matrix training	GPU	Matrix multiplication applies the same operation to huge data sets, so thousands of GPU cores exploit the data parallelism (SIMD).
c) Fitness watch	RISC processor	Simple, fixed-length instructions need fewer transistors and less power, extending battery life.
d) 800 TB off-site archive	Magnetic tape	Lowest cost per GB at huge capacity; slow serial access is acceptable because data is almost never read back.
e) Video journalist	Flash storage (SSD)	No moving parts, so it survives being dropped, and fast read/write speeds suit large daily transfers.
f) Office desktop	Von Neumann multicore CPU	A general-purpose machine holds different programs and their data in the same shared memory; multiple cores run several varied applications at once.

9. FDE register transfers — order 1 — MAR $\leftarrow$ PC 2 — PC $\leftarrow$ PC + 1 3 — MDR $\leftarrow$ memory[MAR] 4 — CIR $\leftarrow$ MDR 5 — Decode: CU splits the contents of the CIR into opcode and operand 6 — Execute: ALU carries out the operation; result to ACC (*Accept steps 2 and 3 either way round / concurrent: the PC may be incremented while the memory read is in progress. The decode and execute steps must come after CIR $\leftarrow$ MDR, in that order.*)

Section B — model answers

Q1 (1 mark, AO1) *Model answer:* The MDR temporarily holds the data or instruction that has just been fetched from memory, or that is about to be written to memory. *Mark point:* holds data/instructions being transferred to or from memory [1].

Q2 (2 marks, AO1) *Model answer:* Pipelining overlaps the stages of the fetch–decode–execute cycle for consecutive instructions, so while one instruction is being executed the next is being decoded and a third is being fetched. This keeps every part of the processor busy at once, so more instructions complete per unit time (higher throughput), even though each individual instruction takes just as long. *Mark points:* stages of FDE for different instructions are overlapped / instruction 2 is fetched while instruction 1 is decoded [1]; so throughput increases / the processor completes more instructions per second because no part of the CPU sits idle [1].

Q3 (3 marks, AO2) *Model answer:* A barcode scanner (input) to read the barcode on each product so its price can be looked up; a touchscreen (input and output) so the customer can see their itemised bill and select options such as paying or using their own bag; a receipt printer (output) to give the customer a printed record of the transaction. *Mark points:* one mark per device **with a purpose linked to the checkout scenario** (max 3). Accept e.g. barcode scanner, touchscreen, receipt printer, electronic scales (to weigh items in the bagging area/loose produce), card reader/contactless reader (to take payment), speaker (to give audio prompts such as "unexpected item"). A device named with no scenario-linked purpose gets no mark.

Q4 (4 marks, AO2) *Model answer:* First, an SSD has no moving parts, whereas an HDD relies on spinning platters and a moving read/write head; because the photographer carries the drive in a rucksack in the field, an HDD is likely to be damaged by knocks and drops, while the SSD will survive them, protecting her irreplaceable photos. Second, an SSD has much faster write speeds than an HDD; since she transfers thousands of large image files every day, the transfers will finish far more quickly, saving her significant time on location. *Mark points* (2 × 2): durability — no moving parts / solid state [1] linked to being carried/knocked about in a rucksack in the field [1]; speed — much faster read/write than a mechanical disk [1] linked to the thousands of large files transferred daily [1]. Also accept: lower power draw [1] linked to working on location away from mains power [1]; smaller/lighter [1] linked to portability in the rucksack [1]. Max 4.

Q5 (4 marks, AO2)

Register	Contents after fetch
PC	0065
MAR	0064
MDR	ADD 0100
CIR	ADD 0100

Mark points: one mark per correct row [4]. The PC must have been incremented to 0065; the MAR holds the address that was fetched from (0064); the MDR and CIR both hold the fetched instruction ADD 0100 (the instruction has not yet been decoded or executed, so no addition has taken place).

Q6 (3 marks, AO1) *Model answer:* Cache is a small amount of very fast memory located on or next to the CPU. It stores the most frequently and recently used instructions and data. With more cache, more of these items can be held close to the processor, so the CPU finds what it needs in cache more often (more cache hits) and has to make fewer slow fetches from main memory across the bus — so it spends less time idle waiting for data, and performance improves. *Mark points:* cache is small, very fast memory on/near the CPU [1]; holds frequently/recently used instructions and data [1]; more cache means fewer (slow) accesses to RAM / higher hit rate, so the CPU waits less [1].

Q7 (4 marks, AO1) *Model answer:* A RISC processor has a small set of simple, fixed-length instructions, each designed to complete in a single clock cycle, whereas a CISC processor has a large set of complex, variable-length instructions that may each take several cycles. Also, in RISC the complexity is shifted into the compiler (a single CISC instruction may need several RISC instructions), while CISC builds the complexity into the hardware itself, needing more transistors — which is why RISC chips typically use less power and pipeline more easily. *Mark points:* any two differences, each correctly described from both sides for 2 marks: instruction set size/complexity (few simple vs many complex) [2]; instruction length (fixed vs variable) and/or cycles per instruction (one vs many) [2]; where complexity lives (compiler/software vs hardware/transistors) [2]; consequences (RISC pipelines more easily / lower power; CISC needs fewer instructions per program / more compact code) [2]. Max 4.

Q8* (9 marks, AO3) — levels of response

Level descriptors: - **Level 3 (7–9 marks):** Thorough knowledge and understanding of processor performance factors. The discussion is consistently **applied to the studio's dual rendering/office workload**, considers both sides (benefits and limits/cost of each factor), and presents a **well-developed, balanced line of reasoning** leading to a supported recommendation. - **Level 2 (4–6 marks):** Reasonable knowledge and understanding; some application to the scenario, but the argument may be one-sided or only partially developed; a line of reasoning is present but not fully sustained; a recommendation may be given with limited support. - **Level 1 (1–3 marks):** Basic, generic points about clock speed/cores/cache/GPU with little or no application to the scenario; points are made in isolation with no coherent line of reasoning; no meaningful recommendation. - **0 marks:** No creditworthy response.

Model Level-3 answer: Rendering 3D scenes is a highly parallel workload: each frame is made of millions of pixels/polygons that can be processed independently. This means the biggest single gain will come from a **dedicated GPU**, whose thousands of simple cores apply the same operation to many data items simultaneously (data parallelism) — exactly what rendering requires — provided the studio's rendering software supports GPU acceleration. **More CPU cores** would also help, because rendering can be split into parallel tasks across cores; however, the benefit is not linear, since parts of the pipeline are sequential and the cores share memory bandwidth, so beyond a point extra cores give diminishing returns.

A **higher clock speed** raises the number of FDE cycles per second on every task, which benefits both rendering and the office work, but modern chips are limited by heat and power, so large clock increases are expensive for modest gains. **More cache** reduces slow fetches from RAM by keeping frequently used data near the CPU, which helps general responsiveness, but it is the least significant factor here because rendering datasets are far larger than any realistic cache.

On the other side of the balance, the everyday tasks — email and word processing — are light and largely sequential; almost any modern CPU handles them easily, so it would be wasteful to optimise the budget around them.

Recommendation: since the workstations spend most of their time rendering, the studio should spend the bulk of the budget on a capable dedicated GPU plus a mid-to-high core-count CPU, accepting a moderate clock speed and standard cache. This targets the dominant, parallelisable workload while remaining more than sufficient for office tasks.

Indicative content (AO1): clock speed = cycles per second; cores = independent FDE units; cache = fast memory near CPU reducing RAM accesses; GPU = massively parallel SIMD processor. (AO2 *application*): rendering is data-parallel so suits GPU/multicore; office tasks are light/sequential; software must support parallelism/GPU. (AO3 *evaluation*): diminishing returns of cores; heat/power limits of clock

speed; cache limited by dataset size; cost trade-offs; justified recommendation weighted to the dominant workload.